

Cormas: common-pool resources and multi-agent systems

François Bousquet¹, Innocent Bakam², Hubert Proton³, and Christophe Le Page¹

(1) Cirad Green, Campus de Baillarguet, BP 5035, 34032 Montpellier cedex, France
{bousquet, lepage}@cirad.fr

(2) Département d'Informatique, Yaoundé University, Cameroon
ibakam@uycdc.uninet.cm

(3) Laboratoire Systèmes Coopératifs, Ecole des Mines de Saint-Etienne, 158 cours Fauriel, 42023 StEtienne,
France
proton@emse.fr

Abstract

*This paper describes a simulation environment, called *Cormas*, that relies on multi-agent systems and has been achieved in *SmallTalk*, using *VisualWorks* software. Such a simulation tool may prove useful to better understand the complex interactions between natural and social dynamics when studying renewable resource management. The general principles of the *Cormas* platform are first presented, then the implementation is described. Two models built with *Cormas* allow to illustrate the use and the genericity of this tool.*

1 Introduction

Over the last few years, the field of distributed artificial intelligence and multi-agent systems has expanded rapidly, with several communities of researchers carrying out studies in this area [1]. Different approaches have been adopted, with most work focusing on distributed problem solving, though another community of researchers is concentrating on simulation using multi-agent systems. When the aim is to solve distributed problems, we generally use “cognitive” agents with highly developed deliberation and internal control capacities. When we wish to simulate populations of agents, emphasis is generally placed on the notion of environment. We examine the relations between agents via their actions on the environment. Ecologists and social scientists are today contributing to the development of multi-agent simulation methods via a wide variety of applications and several methodological studies [2] [3]. The *Green* (Renewable Resource Management and Environment) research team is studying renewable resource management. To this end, it is seeking to understand the interactions between natural and social dynamics. To understand the complexity of these interactions, we use a simulation methodology which makes use of multi-agent systems. Models have been developed on the theme of irrigated land management in Senegal [4], wildlife management in the forests of East Cameroon [5] [6] [7], fuelwood management [8], herd mobility in the Sahel [9] and potlatch type exchange structures [10]. In association with these various research projects, tools have been developed and a modelling approach has been proposed [11]. To bring these tools together, we have developed a simulation environment called *Cormas* (Common-pool resources and multi-agent system). In this article we will present this simulation environment which aims to simplify the task of resource management simulation. This environment was built using *VisualWorks* software. It uses and proposes *SmallTalk* as a development language. We will first present the principles of this environment and then the implementation that we propose. Two examples will then be given to illustrate the use of this platform. One concerns the simulation of agents moving in space while the other concerns a simulation of communicating agents drawing up land exchange contracts.

2 Principles

There are a large number of environments for creating multi-agent systems. Some environments, such as *JavaNetAgents* [12], *Madkit* [13] and *Lalo* [14], are oriented towards communication between distributed systems, while others are oriented towards system simulation. Examples of the latter include the Cellular environment which is based on the concept of a cellular automaton or the *Swarm* environment developed in Santa-Fé. The principles of the *Cormas* environment are expressed by different choices. Three types of program are created. The first set of programs concerns the agents and the interactions between agents in space or by communication. The second set of programs concerns overall control of the dynamics while the third set seeks to define an observation of the system via the viewpoint concept.

2.1 Agents and interactions

We have defined two types of agent. The first type is the situated agent. An entity is said to be situated if its position is a relevant item of data in the model. When an entity is declared to be situated, it possesses a spatial reference and is characterized by its field of perception. The second type of agent is the communicating agent. An entity is said to be communicating when it can communicate with another entity which is not located in direct proximity. Communication must not be perceived simply as an exchange of information. In the context of renewable resource management, gifts, exchanges and contracts are all situations which involve communication between entities. The third type of agent is the situated and communicating agent which corresponds to the sum of the two previous ones. We are currently working on a fourth type of agent which is the spatial agent that we will mention in the paragraph on future developments. There are two modes of interaction. The first concerns the way in which space is taken into account. We have created a spatial environment containing the situated agents. By space, we mean physical space (territory, zone, region). The space is divided into cells or patches linked by relations of proximity. The cell is the smallest referenced spatial unit. To play its role as a medium, as an intermediary between agents, the patch possesses a list of its occupants and implements primitives which enable communication to take place. It also provides primitives for access to its vicinity (immediate or extended). The second type of interaction involves the notions of message and transmission channel. Communicating agents can send each other messages through a channel. The channel models a communication path between agents who are not necessarily neighbours. The channel implements message transmission primitives between connected agents.

2.2 Controlling the model dynamics

Agents, space cells and channels are the basic components required to prepare a multi-agent simulation. To perform a simulation, it is first necessary to create an artificial world in a given configuration. A space must be created and its dimensions defined, along with the number of agents required and the state in which they exist. The time dynamics of interactions between agents must then be specified. This poses scheduling problems. For this reason, in the case of simulations, we isolate the control part of the dynamics. It is as important for the modeller to define the control as it is to define the behaviour of agents. Indeed, the different types of control determine the sensitivity of simulations.

2.3 The observer's viewpoints

Emergence is one of the founding concepts of multi-agent systems. It involves simulating behaviors and interactions at the microscopic level in order to observe the emergence of complex behaviors on a macroscopic level. Progressively, less importance is being given to the problem of emergence and more to that of observation. The *Cormas* environment is equipped with several tools to define the observation of simulations. Firstly, we introduced the notion of viewpoint. Hence, a portion of space or an agent will be observed from different viewpoints. The *Cormas* user is thus both a modeller and an observer. He must therefore define his observation and implement it, in the same way as he programs agents and the control of dynamics.

3 Implementation

Figure 1 presents the main interface of the *Cormas* simulation environment. In the first part of the modelling process, the agents and types of interaction are implemented via the space or the messages and the channel. Then, the control of dynamics and observation are implemented before opening the visualization interfaces. Implementations mainly involve the preparation of *SmallTalk* language classes inheriting predefined classes.

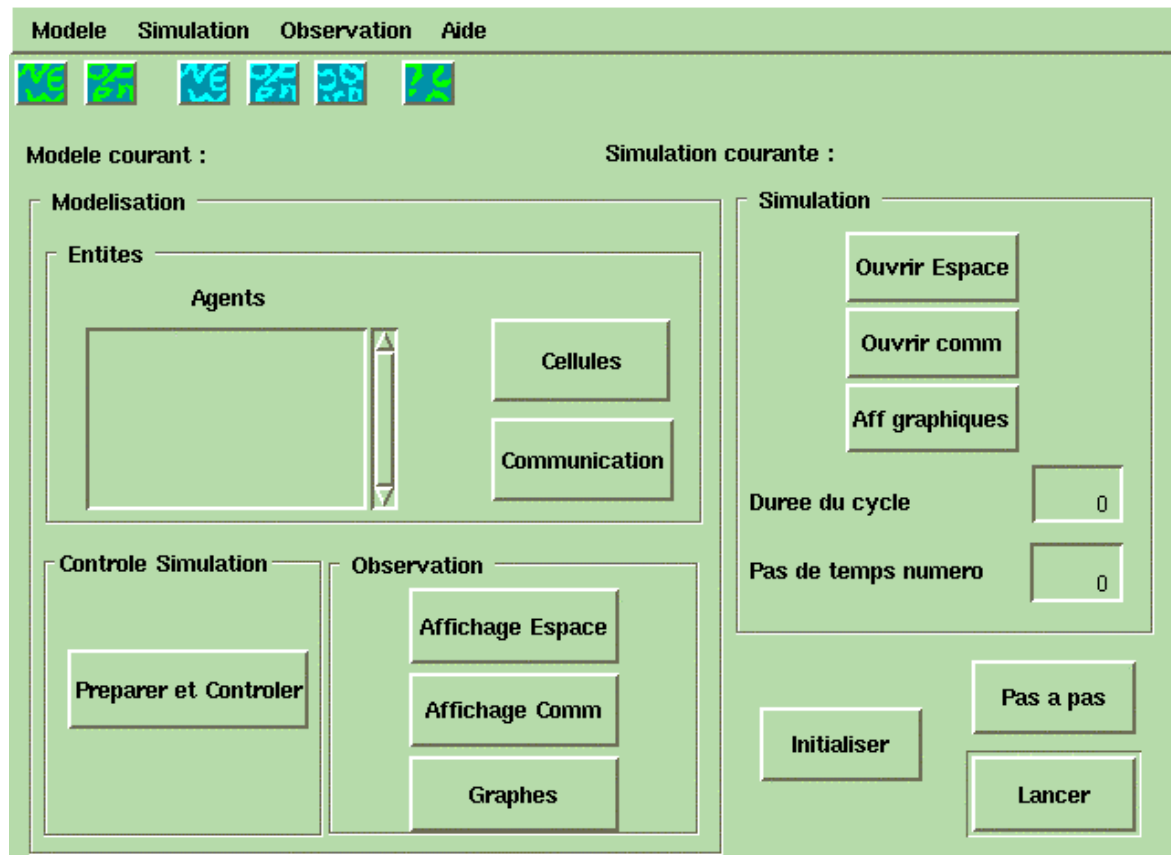


Fig. 1. *Cormas* main interface

3.1 Implementation of interactions via space and the communication channel

An interface is used to enter the definition of the classes comprising the cell. Each model implements, if appropriate, its space cells. A *Patch* class is preprogrammed and proposed for the inheritance. This class includes attributes which define the vicinity (ordered or not, in a variable perception radius). Each patch possesses a *lesOccupants* attribute which automatically contains the list of agents located upon it, classified by type of agent. The user can add attributes or methods which initialize or modify the state of the cell. It is thus possible, for example, to model a cellular automaton by implementing only the automaton transition function. The communication button opens an interface which defines message classes. A *Message* class is preprogrammed and proposed for the inheritance. The agents send each other instances of these classes. By default, three preprogrammed attributes can be used to identify the emitter, the receiver and the symbol of the message. The user can add the attributes required for communication between agents.

3.2 Implementation of agents

The user possesses an interface to create classes representing agents. The class created can inherit three types of object. For the *ObjetsSitué* class, the patch attribute is preprogrammed. Two methods are preprogrammed. The depart method is broken down into two actions. First, it deletes the agent

from the list of agents on the patch, then it empties the agent's patch attribute. The arrive method is symmetrically broken down into two actions. It is a method whose argument is constituted by the patch upon which the agent arrives. The first action is therefore to record the patch in the agent's patch attribute. The second action is to record the agent in the list of agents present on the patch. The modeler's task is therefore to define how an agent selects a new place. The *ObjetCommunicant* class possesses a channel (*Canal*) attribute. The agent who inherits the *ObjetCommunicant* class is directly linked up to a channel for transmission of messages. The channel is an object that the user does not create. It is created automatically. Likewise, the *Canal* object is connected to all agents inheriting the *ObjetCommunicant* class. The *mailBox* attribute is the agent's mailbox. Several methods are preprogrammed. They allow the messages in the mailbox to be read in different ways: first message, last message, sorting of messages. To program individual control of the agent, the user must program the interpretation of messages. The *ObjetSitueCommunicant* class cumulates the characteristics of the two previous classes to create agents which are both situated and communicating.

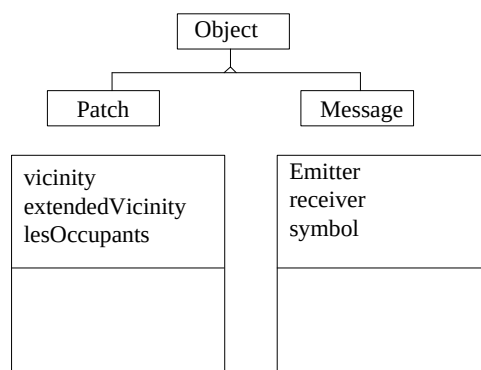


Fig. 2. Modes of interaction: space and messages

3.3 Implementation of the control of simulation dynamics

Once all the basic elements have been created, it is necessary to define the initialization and the way the simulation is to proceed. A class called *Modèle* is created automatically. Each time an agent or space is created, a new attribute of the model (*Modèle*) class is created. These attributes constitute the list of agents or cells created. Hence, the *Modèle* class can control all entities. A certain number of prepared programs can be used for synchronous and asynchronous simulations, for simulations with or without an agent activation order and for several coordination protocols such as net contracts or bidding. The user can thus choose an existing program or make his own program to control the dynamics of agents.

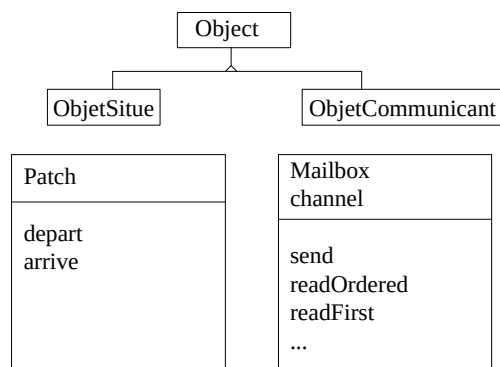


Fig. 3. The agents of *Cormas*

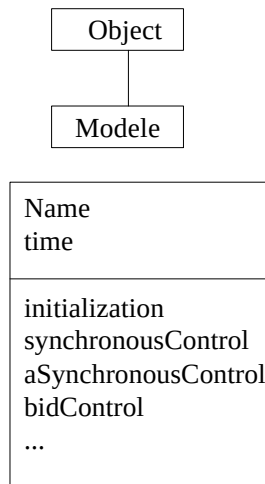


Fig. 4. Control of the simulation

3.4 Implementation of observation

Three types of observation are proposed. First, it is possible to create counters which automatically generate scientific graphs to follow the evolution of variables or quantities (number of agents or cells in one or other state). It is then possible to select the types of communication that the user wishes to observe. An interface can be used to write programs which condition the observation of messages on the communication interface. Lastly, the user can program methods which define the conditions of observation of agents and space cells. Colors are attributed to the various states thus defined.

3.5 Interfaces

Three interfaces are proposed: scientific graphs, a grid to visualize all the cells and situated agents. The number, size and shape of cells can be parameterized. It is also possible to import data files in text mode defining the value of the various cell attributes. Data from *GIS* can also be imported, thus providing actual maps containing several layers of information. The third layer is a link observer which is used to observe the communicating agents and communication graphs which structure the networks [15].

4 Examples

Here we will present two simulation examples implemented with the *Cormas* environment. They do not constitute a real scientific experiment, but small didactic programs. The first illustrates the notion of mobile agents via a simulation of predator agents pursuing prey agents. The second is a simulation of land regrouping which presents two algorithms of coordination between communicating agents.

4.1 The pursuit model

To model a predator chasing its prey, we created two agent classes which inherit the *ObjetSitué* class. They thus inherit the arrive and depart methods. To simulate the chase, we program the method which chooses the place of arrival of the predator (*Predateur*) agent or of the prey (*Proie*) agent. For example, we can implement a method in which the predator agent chooses a cell within a perception radius containing a prey agent. In accordance with what is stated above, for the agent this means choosing a cell containing the prey agents in the vicinity (*voisinageEtendu*) of the patch where it is located. A code line carries out an instruction to identify this type of cell.

```
self patch voisinageEtendu select:[a] (a lesOccupants at:#Proie) isEmpty not]
```

The code line which, for the prey agent, will identify a cell where there are no predator agents is:

```
self patch voisinageEtendu select:[:a| (a lesOccupants at:#Predateur) isEmpty]
```

All the key words of these two code lines are preprogrammed (*patch*, *voisinageEtendu*, *lesOccupants*). Simply by writing these two code lines, we can simulate a predator chasing its prey. The predator class must also implement an eat method which involves detecting the prey agents present in the cell and thus activating their depart method. After implementing the agents' methods, a simulation control method must be implemented. Various implementations are possible, depending on whether the predators act first, the prey act first or whether the two lists are mixed before they are made to act.

4.2 Land regrouping

In this exercise we simulate a land regrouping process. A spatial grid is created, with each cell representing a plot of land. *Farmer* agents are created as *ObjetsSituésCommunicants*. The farmer agents are placed randomly on the grid. When the system is first created, the plots are attributed randomly to the farmers. At each simulation step, all the plots are offered for rent to all the farmer agents. To this end, a *MsgPlot* class is created including the amount attribute and the object attribute in addition to the preprogrammed attributes (*emitter*, *receiver*, *symbol*). To simulate conversations between agents discussing rental contracts, the symbol attribute can take four different values (*#propose*, *#accept*, *#finalize*, *#sorry*).

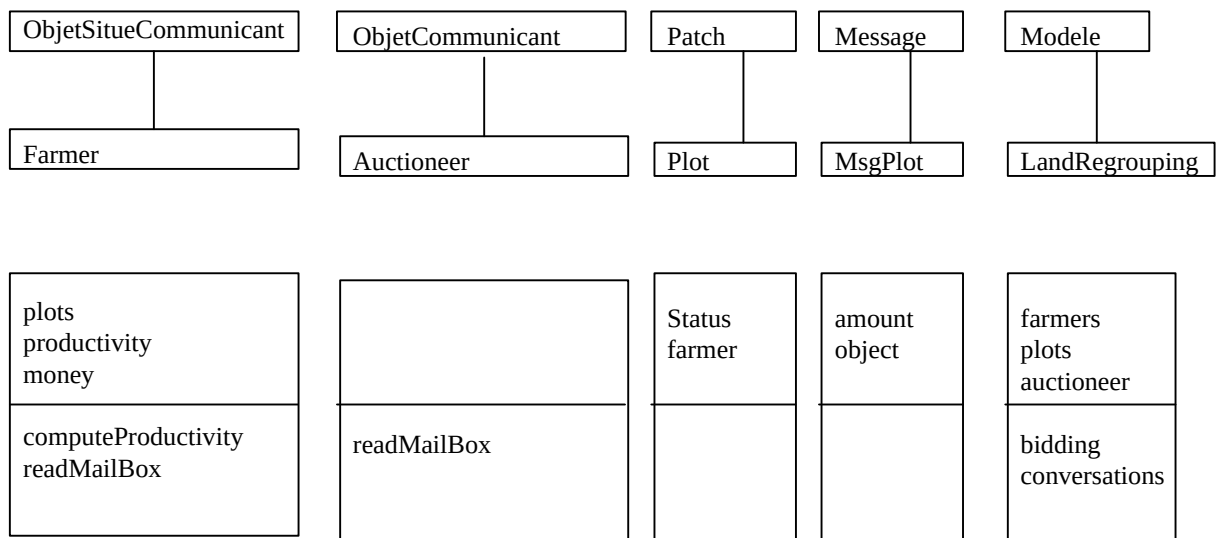


Fig. 5. Land regrouping agents

The amount attribute will be a proposed or accepted value and the object attribute indicates the plot concerned by the transaction. The farmer agents calculate the productivity of a plot according to the distance between the plot and the place they are situated. This productivity calculation enables them to define the amount proposed for rental of the plot.

The first algorithm corresponds to a bidding process. An auctioneer (*CommissairePriseur*) agent is created as an *ObjetCommunicant*. A method of the auctioneer object enables him to send proposals to all farmer agents. Another of the auctioneer's methods enables him to sort all the *#accept* type messages in increasing order of the amount proposed. He sends the *#finalize* method to the agent who offers the best price and the *#sorry* message to all the others. The farmer agents read the messages in their mailbox and send messages of the type *#accept* to offer a rental amount. To organize this simulation, the following control method is used:

1. The auctioneer offers all plots to all farmers
2. The farmers make bids
3. The auctioneer calculates the best bids and sends *#sorry* and *#finalize* type messages
4. The farmers calculate their income

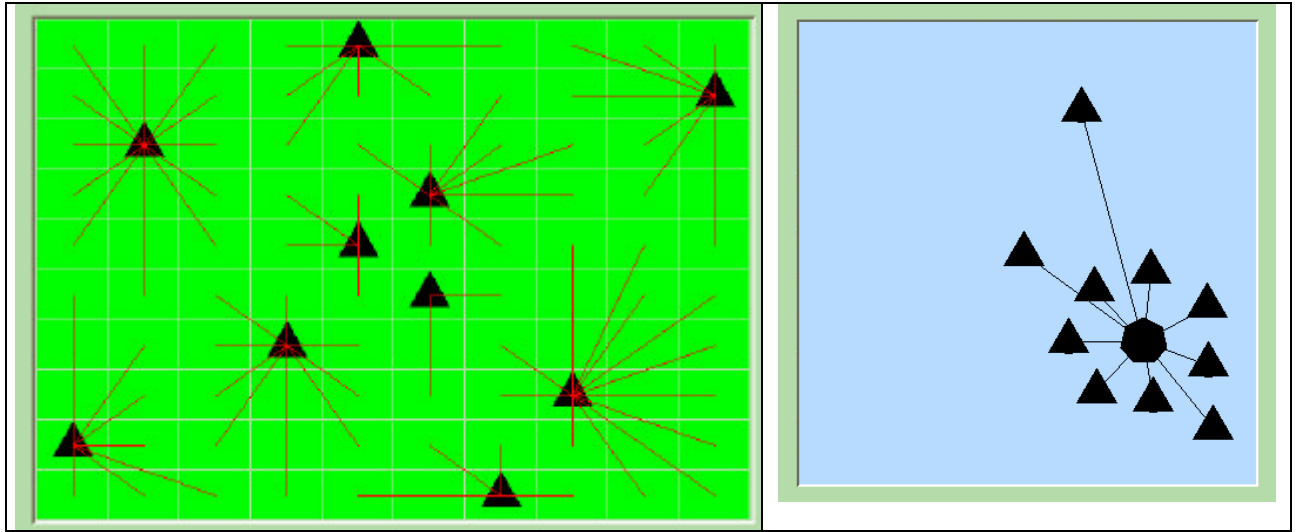


Fig. 6. Space organization and communication graph with an auctioneer

Figure 6 presents the two interfaces. The spatialized interface shows that the plots have been attributed to the closest farmer. This attribution is very stable. The existence of the auctioneer makes it possible to compare the bids and select the best ones. The link visualization interface shows the communication graph organized around the auctioneer agent.

The second algorithm simulates direct relations between farmer agents. There is no auctioneer. Each farmer agent offers all his plots to all other farmer agents who, if interested, send back an *#accept* message. The proposer sends back a *#finalize* type message if the offer suits him and a *#sorry* type message if he does not want to take up the offer or if the plot has already been rented out. The agent programming is based on a method of mailbox reading.

- If the message is a *#proposition* symbol then calculate the productivity of the proposed plot and send back an *#accept* type message with the calculated sum
- If the message is *#accept* symbol, then if the plot is already rented out or if the proposed amount is insufficient, send back a *#sorry* symbol message, otherwise send back a *#finalize* message
- If the message is *#finalize*, then cultivate the plot
- If the message is *#sorry*, then recover the proposed amount.

To organize this simulation, the following dynamics control method is used

1. All farmers offer their plots for rent
2. So long as the agents have messages to read, activate the agents
3. The agents calculate their income.

In this imulation there is very little overall control. The agents are left to communicate until they have all finished. The order of communications is not controlled. Figure 7 presents the two interfaces. The spatialized interface shows that the attributions are much less structured and much less stable. The plots have not been attributed to the closest farmer. The attribution depends on the scheduling of the conversations. Indeed, the evolution of dynamics depends upon the agreements reached between agents. The link visualization interface shows that the communications graph is complex.

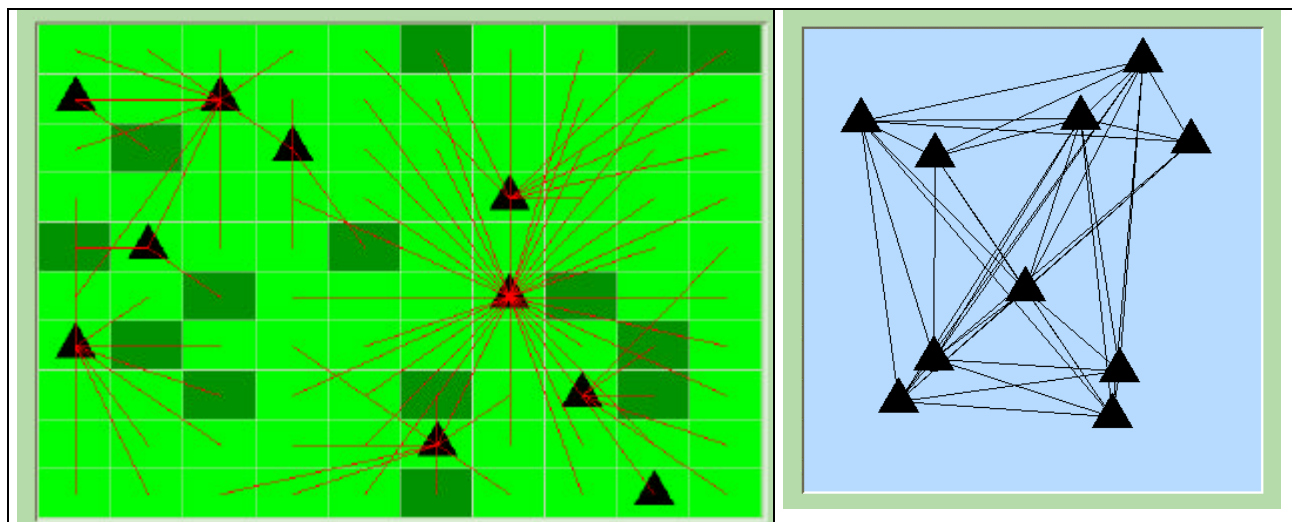


Fig. 7. Space organization and communication graph: direct relations between farmer agents

5 Future developments and conclusion

After several years of dedicated multi-agent simulations on the management of renewable resources, we have now developed a simulation environment. This environment can be used for rapid creation of situated and communicating agents. Our application also led us to focus on the problem of common spaces and interaction networks. Many promising avenues of research are now opening for the improvement and further development of this simulation environment. Firstly, we are currently creating a new basic entity, the spatialized object, which is made up of a set of spatial cells. The spatialized objects thus corresponds to the notion of a group capable of communicating and exchanging cells. Control of spatial dynamics will thus depend upon the relation between the behavior of the basic entity, i.e., the cell, and a group of cells which may have a behavior. This environment does not focus on the agent's internal deliberations. We are nevertheless interested in the problems of beliefs, memories, representations and deliberations. We will therefore concentrate on developing tools to model the internal control of agents more easily. We will also continue to develop the three modelling components represented in *Cormas*: agents and interactions, control of dynamics and observation viewpoints.

References

1. Ferber, J.: Les Systèmes Multi-Agents. Vers une intelligence collective. InterEditions, Paris (1995)
2. Bousquet, F.: Usages des ressources renouvelables et modélisation des représentations: une approche par les systèmes multi-agents. In: Blasco, F. (ed.): Tendances nouvelles en modélisation pour l'environnement. Elsevier, Paris (1997) 187-193
3. Treuil, J.P., Perrier, E., Cambier, C.: Directions pour une approche multi-agents de la simulation de processus physiques spatialisés. In: Quinqueton, J., Thomas, M.C., Trousse, B. (eds): Actes des 5^{èmes} journées francophones sur l'IAD et les SMA. Hermès, Paris (1997) 211-228
4. Barreteau, O., Bousquet, F., Weber, J.: Modes de gestion et viabilité des périmètres irrigués: questions de représentation. In: Blasco, F. (ed.): Tendances nouvelles en modélisation pour l'environnement. Elsevier, Paris (1997) 153-159
5. Takforyan, A.: Vers une gestion locale de la faune sauvage en Afrique ? le cas de l'Est-cameroun. In: Gestion communautaire des ressources naturelles renouvelables et développement durable. Colloque international Harare (1996)

6. Proton, H.: Systèmes multi-agents pour l'étude de l'organisation d'une société de chasseurs et d'agriculteurs dans le cadre des ressources renouvelables. Master's thesis, Université Montpellier II (1996)
7. Bakam, I., Takforyan, A., Proton, H., Bousquet, F., Weber, J.: Simulations spatio-temporelles d'interactions entre chasseurs et ressources par les systèmes multi-agents: illustration par le cas de la chasse dans l'Est-cameroun. In: Gestion de la faune sauvage et usages de l'espace dans les paysages ouverts. Actes de Colloque, Lyon (1997)
8. Bousquet, F., Guizol, P., Ndikumagenge, C., Proton, H.: Multi-agent models for analysing dynamics between forest and agroforest on a regional level. *Agroforestry System*, in revision
9. Bah, A.: Simulation multi-agents des modes d'utilisation des ressources en propriété commune; le cas de la mobilité pastorale en zone intertropicale sèche. Master's thesis, Ecole Supérieure Polytechnique de Dakar (1997)
10. Rouchier, J.: Le potlatch: une modélisation par les systèmes multi-agents. Master's thesis, Université Orléans (1996)
11. Bousquet, F., Barreteau, O., Mullon, C., Weber, J.: Modélisation d'accompagnement: Systèmes multi-agents et gestion des ressources renouvelables. In: Quel environnement au XXIème siècle ? Environnement, maîtrise du long terme et démocratie. Proceedings of a workshop at the Abbaye de Fontevraud (1997)
12. Merlat, W., Seyrat, C.: JavaNetAgents: une plateforme d'exécution d'agents mobiles pour le développement des SMA sur internet. In: Quinqueton, J., Thomas, M.C., Trousse, B. (eds): Actes des 5èmes journées francophones sur l'IAD et les SMA. Hermès, Paris (1997) 83-84
13. Gauvin, D., Marchal, H., Saldanha, C.: LALO: un environnement de programmation ouvert pour les systèmes multi-agents. In: Quinqueton, J., Thomas, M.C., Trousse, B. (eds): Actes des 5èmes journées francophones sur l'IAD et les SMA. Hermès, Paris (1997) 91-92
14. Gutknecht, O., Ferber, J.: MadKit: Organizing heterogeneity with groups in a platform for multiple multi-agent systems. Rapport Interne LIRMM 97188 (1997)
15. Proton, H., Bousquet, F., Reitz, P.: Un outil pour observer l'organisation d'une société d'agents. le cas d'une société d'agents chasseurs-agriculteurs. In: Quinqueton, J., Thomas, M.C., Trousse, B. (eds): Actes des 5èmes journées francophones sur l'IAD et les SMA. Hermès, Paris (1997) 159-172